MODULE 14 · DOMAIN 2 - ML MODEL AND FM DEVELOPMENT

Training jobs and distributed training

Script mode, parallelism, Spot, checkpoints and warm pools.

MODULE 14 of 43 DOMAIN 2 - Model Dev WEIGHT 24%

admin@sthithapragna.com

A training job is a container that starts, reads S3, writes S3, and dies.

What happens when you call fit

01
Provision
Instances start. Billing starts here, not at epoch one.

02
Pull
The container image comes down from ECR.

03
Stage
Data arrives by the input mode from module 02.

04
Train
Your script runs. Metrics stream to CloudWatch.

05
Save
Output copied to S3, then instances terminate.

ON THE EXAM

Steps one to three are the startup cost. Warm pools attack step one; FastFile and FSx attack step three.

Where the time actually goes

- **Billable seconds** covers the whole job: provisioning, staging, training and saving.
- **Training seconds** is the subset doing actual learning. The gap is pure overhead.
- On Spot jobs, SageMaker AI reports **savings** alongside these, so the discount is measurable.
- A large gap between the two numbers is the signal to look at input mode or warm pools.
- Both figures appear on the job description and in the console.

ON THE EXAM

This is the diagnostic for the whole cost half of the module: billable time minus training time is what you are trying to shrink.

How much code do you bring?

APPROACH	WHAT YOU SUPPLY
Built-in algorithm	Hyperparameters only. No code at all
Script mode	Your training script. AWS supplies the framework container
Own container	Your image in ECR. AWS supplies infrastructure only

ON THE EXAM

Script mode is the middle path and the one most teams live in. You own the model code, not the environment.

Script mode, and what the container gives you

- AWS maintains the image: framework version, CUDA, drivers, the distributed libraries.
- Your data arrives at **local paths**, given by environment variables such as SM_CHANNEL_TRAIN.
- Anything written to **SM_MODEL_DIR** is packaged and copied to S3 as the model artefact.
- Hyperparameters arrive as **command-line arguments** to your script. That is how tuning drives it.
- Extra Python packages go in a requirements file, installed at container start.

> ON THE EXAM
>
> The environment variable contract is the whole interface. Read from SM_CHANNEL paths, write to SM_MODEL_DIR, and the rest is handled.

There are two different reasons to use more than one GPU, and they need **different answers**.

Data parallel or model parallel?

DATA PARALLEL, WHEN

Training is too slow

- The model fits on one GPU comfortably.
- Every GPU holds a full copy of it.
- Each processes a different data shard.
- Gradients are synchronised each step.

MODEL PARALLEL, WHEN

The model does not fit

- One GPU cannot hold the model at all.
- The model itself is split across GPUs.
- Each GPU holds some layers or some tensors.
- Activations pass between devices.

THE TIEBREAKER

Data parallel does nothing for an out-of-memory error, because every GPU still holds a complete copy of the model.

Data parallel, and the SMDDP library

- Each instance trains on its own shard, then all gradients are combined and applied.
- The bottleneck is **communication**, not compute. Gradients cross the network every step.
- **SMDDP** is a collective communication library optimising **AllReduce** and **AllGather** for AWS.
- It exploits AWS network infrastructure and EC2 instance topology, so scaling stays closer to linear.
- Pair it with ShardedByS3Key input distribution from module 02, so each instance reads its own slice.

> ON THE EXAM
>
> Doubling instances does not halve training time. Communication overhead is why, and SMDDP exists to reduce it.

EFA and instance topology

- **Elastic Fabric Adapter** gives low-latency, high-throughput networking between instances.
- For gradient synchronisation every step, network latency is the limiting factor. EFA attacks it directly.
- Supported on the larger P and Trainium instance types, not on small ones.
- SMDDP is written to exploit instance topology, so placement matters to throughput.
- Scaling out on instances without fast interconnect is where near-linear scaling stops being near-linear.

ON THE EXAM

Distributed training is a networking problem wearing a machine learning costume. That is why the instance type matters so much.

Choosing the instance count

- Scaling is **never perfectly linear**. Sixteen instances do not give sixteen times the speed.
- Efficiency falls as you add nodes, because communication grows while per-node work shrinks.
- **Measure it:** run at 1, 2 and 4 instances and look at where the speed-up curve bends.
- Fewer larger instances often beat more smaller ones, because some traffic stays inside one machine.
- AWS benchmarks SMDDP with near-linear scaling up to 128 nodes on suitable instances.

ON THE EXAM

Total cost is roughly instances times time. If doubling instances gives a 1.5x speed-up, you have just made it more expensive.

Distributing changes your batch size

- **Overall batch size scales linearly with instance count.** Eight instances means eight times the batch.
- A larger batch changes the optimisation problem. The old learning rate is now wrong.
- **AWS states this explicitly:** hyperparameters such as learning rate must be revisited to ensure convergence.
- The usual response is to scale the learning rate with the batch, often linearly.
- Symptom if ignored: the model trains faster and converges worse, or not at all.

> ON THE EXAM
>
> 'We scaled out and accuracy dropped' is this, almost every time. Adding instances is not a free speed-up.

Model parallel, and SMP v2

- **SMP v2** is integrated with **PyTorch FSDP**, so you use familiar APIs rather than a bespoke one.
- **Tensor parallelism** splits individual layers across devices.
- **Pipeline parallelism** puts different layers on different devices and streams batches through.
- **Sharded data parallel** splits parameters, gradients and optimiser state across the group.
- That third one is the pragmatic middle ground, and it is what FSDP and DeepSpeed ZeRO do.

ON THE EXAM

Sharded data parallel is the answer more often than pure model parallel: it cuts per-GPU memory without redesigning the model.

Fixing out-of-memory, cheapest first

- **Reduce the batch size.** Free, immediate, and it changes the optimisation slightly.
- **Gradient accumulation:** small batches accumulated to simulate a large one. Memory of small, behaviour of large.
- **Mixed precision**, such as bfloat16, roughly halves activation memory.
- **A larger GPU instance.** Costs money, changes no code.
- **Sharded data parallel, then full model parallel.** Most capable, most complexity.

> ON THE EXAM
>
> Work down this list before reaching for model parallelism. Most out-of-memory errors are solved in the first three rows.

Which parallelism

THE SYMPTOM	WHAT TO REACH FOR
Training takes days, model fits fine	Data parallel, with SMDDP
Out of memory on one GPU	Sharded data parallel first, then model parallel
Model has billions of parameters	SMP v2 with tensor or pipeline parallelism
Scaling out, but speed-up is poor	Communication bound. SMDDP, or larger instances
Accuracy fell after scaling out	Batch size grew. Retune the learning rate

ON THE EXAM

The first two rows are the whole module in miniature: speed is data parallel, memory is model parallel.

Open-source distributed training

- **PyTorch FSDP** and **DeepSpeed ZeRO** both shard training state, and both run on SageMaker AI.
- **Hugging Face** containers and the Accelerate library are supported first-class.
- SMP v2 **integrates with FSDP** rather than replacing it, so this is not an either-or choice.
- SMDDP can act as the **communication backend** underneath FSDP or DeepSpeed.
- So you keep portable open-source code and still get the AWS network optimisation.

> ON THE EXAM
>
> A useful framing: the AWS libraries optimise the layer below your framework, not the framework itself.

SageMaker HyperPod

- **Persistent clusters** for very large training runs, rather than ephemeral job instances.
- Built for jobs that run for **weeks**, where a node failure would otherwise restart everything.
- **Automatic fault recovery:** it detects a failed node, replaces it, and resumes from the last checkpoint.
- Supports both **SMDDP and SMP**, and orchestration with Slurm or Kubernetes.
- It is for foundation-model-scale training. Overkill for a nightly XGBoost job.

> ON THE EXAM
>
> The signal is duration and cluster size. Weeks of training across many nodes makes node failure a certainty, not a risk.

Training cost is instance price times time. There are levers on both.

Managed Spot training

- Uses spare EC2 capacity at a **large discount**, often the majority of the instance price.
- AWS can **reclaim the capacity** at any time. Your job is interrupted.
- SageMaker AI handles the interruption and **resumes the job** when capacity returns.
- **Only if checkpoints exist.** Without them it restarts from the beginning.
- Set **MaxWaitTimeInSeconds** above MaxRuntimeInSeconds to allow for waiting.

> ON THE EXAM
>
> Spot without checkpointing is not cheap training. It is a lottery on whether the job finishes before an interruption.

The two timeouts, and what each does

SETTING	MEANING
MaxRuntimeInSeconds	How long the job may actually train before being stopped
MaxWaitTimeInSeconds	Total time allowed including waiting for Spot capacity
The rule	For Spot, MaxWait must be greater than or equal to MaxRuntime
MaxPendingTimeInSeconds	How long to wait for resources before giving up entirely
Why it matters	A runaway job with no limit bills until somebody notices

ON THE EXAM

Setting a max runtime is cheap insurance. A job that hangs on a data-loading deadlock will otherwise run to the service limit.

Checkpointing

- Your script periodically writes model state to a **local checkpoint path**.
- SageMaker AI syncs that path to **S3**, and restores it when a job resumes.
- On resume, your script must **look for an existing checkpoint and continue from it**.
- A script that always starts from scratch makes checkpointing pointless, silently.
- Checkpoint frequency is a trade: more often means less lost work and more I/O.

ON THE EXAM

Checkpointing is a two-sided contract. SageMaker AI restores the files; your code has to decide to use them.

Reproducing a training run

- Record the **container image URI**. 'Latest' is not a version, and it moves.
- Record the **hyperparameters**, the **data location**, and the **dataset version** from module 06.
- Set a **random seed**, or two identical configurations produce two different models.
- Distributed training adds non-determinism of its own, so exact reproduction is harder at scale.
- SageMaker AI records job metadata automatically. Experiments and MLflow organise it. Module 19.

> ON THE EXAM
>
> 'It worked last month' is not reproducible unless the image, the data and the seed were all recorded.

Managed warm pools

- Retains provisioned infrastructure after a job finishes, so the next job skips cold start.
- Set **KeepAlivePeriodInSeconds** in ResourceConfig. **Maximum is 3,600 seconds**, sixty minutes.
- Reuse requires **matching specifications**, such as instance type and count.
- **Warm pools are billable** while retained. Retained time times instance count.
- Terminate early by updating KeepAlivePeriodInSeconds to zero.

ON THE EXAM

The case is iterative experimentation: many short similar jobs, where provisioning is a large share of each one.

Warm pool lifecycle

STATUS	MEANING
InUse	A training job is running on this warm pool right now
Available	Idle, waiting for a matching job, and still billing
Reused	It moved to a matching job. That pool's lifecycle ended
Terminated	Gone. Time expired, a user ended it, or a patch update

ON THE EXAM

Available is the one to watch. It means you are paying for instances that are doing nothing at all.

Instance selection for training

- **C family** for CPU-bound classical algorithms. **M family** for balanced work.
- **R family** when memory is the constraint, such as large in-memory datasets.
- **P and G families** for GPU training. P for large deep learning, G for lighter or inference.
- **Trainium** instances are purpose-built for training, with a lower cost per unit of throughput.
- A faster instance that finishes in a third of the time is often **cheaper**, not dearer.

> ON THE EXAM
>
> Compare total job cost, not hourly rate. Hourly rate is the number that misleads people most reliably.

VPC mode, isolation and encryption

- A training job can run **attached to your VPC**, so traffic stays on your network.
- **Network isolation** goes further: the container gets no network access at all, inbound or outbound.
- With network isolation on, the container cannot download packages at runtime. Bake them into the image.
- **Inter-container traffic encryption** protects gradients moving between instances in a distributed job.
- It costs some throughput, which matters most on exactly the jobs that need it. A real trade-off.

> ON THE EXAM
>
> Network isolation is the strongest control available and it changes how you build the container. Plan for it, do not enable it late.

Quotas worth knowing about

QUOTA	NOTE
Instances per training job	Limited per instance type. Raisable on request
Concurrent training jobs	A hard stop on how many experiments run at once
Accelerated instance quotas	Often zero by default in a new account. Request them early
Maximum runtime	Service-level ceiling, separate from your MaxRuntimeInSeconds
Hyperparameter tuning jobs	Separate quotas again. Module 15

ON THE EXAM

GPU quotas at zero in a fresh account is the single most common 'the job will not start' cause, and it has nothing to do with your code.

Failure handling and retries

- **Retry strategy** can automatically retry a job that failed for an infrastructure reason.
- It will not fix a bug. Retrying a script with a typo three times costs three times as much.
- With checkpointing, a retry **resumes** rather than restarting. Without it, the retry starts from zero.
- Failure reason is recorded on the job, and the logs persist in CloudWatch.
- In a pipeline, a failed training step should fail the pipeline rather than pass silently. Module 28.

> ON THE EXAM
>
> Retry is for transient infrastructure faults. Distinguish those from code faults before enabling it, or you automate waste.

Local mode and small runs

- **Local mode** runs the training container on your own machine, using the same code path.
- It catches the boring failures fast: bad paths, missing packages, argument typos.
- Then run one short job on a small instance before committing to the full cluster.
- Debugging a syntax error on eight GPU instances is an expensive way to find it.
- Interactive sessions and notebooks serve the same purpose for exploratory work.

ON THE EXAM

Most failed training jobs fail in the first two minutes, for reasons that have nothing to do with machine learning.

Debugger is in maintenance. Profiler is sunsetting

- **SageMaker Debugger** closed to new customers from 30 July 2026. Still named in the exam guide.
- **SageMaker AI Profiler is in the sunset list**, with a termination date set. It is the only ML item there.
- What Debugger did: watched tensors during training and raised rules for vanishing gradients, overfitting, poor GPU use.
- **Answer as the guide expects.** Debugger is still the documented answer for automated training diagnosis.
- In practice: CloudWatch metrics, your own logging, and SageMaker AI MLflow from module 19.

> ON THE EXAM
>
> Same rule as modules 07 and 08. Examinable because the guide names it; not available if you are starting today.

Monitoring training without Debugger

- **CloudWatch Logs** carries everything your script prints. That is your first diagnostic.
- **CloudWatch metrics** give CPU, GPU, memory and disk utilisation per instance.
- **Low GPU utilisation** is the number to watch: it usually means the job is data-starved, not compute-bound.
- Training and validation metrics appear in the console, which is where you read the loss curve.
- Logs persist after the job ends, so a failed job can still be diagnosed.

> ON THE EXAM
>
> GPU utilisation at 20 percent means you are paying for accelerators that are waiting for data. Fix the input path, not the model.

Metric definitions, and why they matter

- In script mode, SageMaker AI scrapes your **stdout** using **regular expressions you supply**.
- Each metric definition pairs a **name** with a **regex** capturing the value your script printed.
- Those named metrics are what **automatic model tuning optimises against**, in module 15.
- A regex that never matches produces no metric, and tuning then has nothing to optimise.
- Built-in algorithms emit their metrics already, so this applies to script mode and custom containers.

> ON THE EXAM
>
> A tuning job that reports no objective metric is almost always a metric definition regex that never matched.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
CUDA out of memory	Batch size too large, or the model needs sharding
Spot job restarts from zero each interruption	Script does not resume from the checkpoint
Accuracy dropped after adding instances	Batch size scaled. Learning rate not revisited
Eight instances barely faster than two	Communication bound, or data loading bound
Job fails seconds after starting	Missing package, bad path, or wrong argument name
Warm pool never reused	Job specifications did not match
Long gap before the loss curve begins	File mode downloading. Module 02

ON THE EXAM

Rows three and four are the distributed-training pair. Neither is a bug; both are consequences nobody planned for.

Training cost levers, ranked

LEVER	EFFECT
Managed Spot	Largest single saving. Requires checkpointing
Right instance family	A GPU job on CPU instances wastes hours; the reverse wastes money
Input mode	FastFile or FSx removes download time you are billed for
Warm pools	Saves provisioning time on short repeated jobs. Billable while idle
Early stopping	Stops a job that has stopped improving. Module 15
Distribution	Faster wall-clock, and rarely cheaper. More instances for less time

ON THE EXAM

Distribution is the one people expect to save money. It buys time, at roughly the same cost or slightly more.

Ask what the job is **waiting on**, and what it would **lose** if it stopped right now.

A research team is training a large language model with roughly 30 billion parameters using PyTorch in script mode. The job fails immediately with a CUDA out-of-memory error on a single GPU instance.

An engineer proposes moving to eight instances with distributed data parallel training, reasoning that eight GPUs give eight times the memory.

What is wrong with that reasoning?

A Nothing. Data parallel training pools GPU memory across instances

B Data parallel replicates the full model on every GPU, so per-GPU memory is unchanged

C Eight instances is below the minimum for distributed training to be effective

D The error is caused by the input mode, not by model size

◆ PAUSE AND DECIDE

What is wrong with that reasoning?

A. Nothing. Data parallel training pools GPU memory across instances

It does not pool memory. Each GPU holds a complete copy of the model, so the same out-of-memory error occurs eight times over.

B. Data parallel replicates the full model on every GPU, so per-GPU memory is unchanged

Exactly the distinction. Data parallel is a throughput solution; this is a capacity problem. Sharded data parallel or SMP v2 is the fix.

C. Eight instances is below the minimum for distributed training to be effective

There is no such minimum, and instance count is not the issue. The model would not fit on eighty instances either, under data parallel.

D. The error is caused by the input mode, not by model size

Input mode affects how data reaches the container. A CUDA out-of-memory error on a 30-billion-parameter model is GPU memory, not data staging.

THE SIGNAL

Speed problem or capacity problem? Data parallel solves the first and cannot touch the second. The error message tells you which you have.

A team runs around forty short training jobs a day while tuning features. Each job trains for about four minutes on two ml.m5.xlarge instances, with identical instance type and count every time.

Reviewing costs, they find provisioning and container pull take roughly three minutes before each job starts training, and they are billed for it.

What should they change?

A Switch to managed Spot training to reduce the instance price

B Move to a single larger instance to remove distributed startup

C Switch the input mode from File to FastFile

D Enable managed warm pools with KeepAlivePeriodInSeconds set

◆ PAUSE AND DECIDE

What should they change?

A. Switch to managed Spot training to reduce the instance price

Cuts the price of the wasted three minutes without removing them. On four-minute jobs, waiting for Spot capacity can cost more time than it saves.

B. Move to a single larger instance to remove distributed startup

Provisioning happens for one instance too. It reduces the cold start slightly and does not address the repeated provisioning at all.

C. Switch the input mode from File to FastFile

Attacks data staging. The stated cost is provisioning and container pull, which happen before any data arrives.

D. Enable managed warm pools with KeepAlivePeriodInSeconds set

Forty short jobs a day with identical specifications is the documented case. Retained infrastructure means later jobs skip the three-minute cold start.

THE SIGNAL

Read which part of the lifecycle is wasteful. Provisioning is warm pools; data staging is input mode; instance price is Spot.

A vision team trains a model for around fourteen hours per run on four GPU instances. Runs happen overnight and there is no deadline before the following morning. Compute cost is the stated concern.

Their training script currently writes the model only once, at the end of the run.

What combination should they adopt?

A. Managed Spot training, as it is, to get the discount immediately

B. Warm pools, to remove provisioning time from each overnight run

C. Add periodic checkpointing with resume logic, then enable managed Spot

D. Move to SageMaker HyperPod for automatic fault recovery

◆ PAUSE AND DECIDE

What combination should they adopt?

A Managed Spot training, as it is, to get the discount immediately

The script saves only at the end, so an interruption at hour thirteen loses everything. Spot without checkpoints is a gamble, not a saving.

B Warm pools, to remove provisioning time from each overnight run

Warm pools save minutes of provisioning on a fourteen-hour job, and the maximum retention is sixty minutes anyway. Negligible here.

C Add periodic checkpointing with resume logic, then enable managed Spot

Long-running, interruption-tolerant, no deadline. That is Spot's case exactly, and checkpointing with resume is what makes it safe.

D Move to SageMaker HyperPod for automatic fault recovery

HyperPod targets jobs running for weeks across large clusters. Fourteen hours on four instances does not need a persistent cluster.

THE SIGNAL

Note the order in option C. Checkpointing first, Spot second. Reversed, the first interruption costs more than the discount saved.

A team moves a PyTorch training job from one instance to sixteen, using distributed data parallel with SMDDP. Wall-clock training time falls from nine hours to under an hour, as hoped.

But the resulting model's validation accuracy is markedly worse than the single-instance version, and on some runs the loss fails to converge at all. No code was changed apart from the distribution configuration.

What is the most likely cause?

A. The overall batch size scaled with instance count, so the learning rate is now wrong

B. SMDDP is incompatible with PyTorch and corrupts gradient synchronisation

C. Sixteen instances exceeds the supported limit for data parallel training

D. Each instance trained on the full dataset, so the model saw the data sixteen times

◆ PAUSE AND DECIDE

What is the most likely cause?

A The overall batch size scaled with instance count, so the learning rate is now wrong

AWS documents this directly: the library scales overall batch size linearly with instances, and hyperparameters such as learning rate must be revisited.

B SMDDP is incompatible with PyTorch and corrupts gradient synchronisation

SMDDP is built for PyTorch and works through torch.distributed. Corrupted gradients would also break training entirely, not degrade it.

C Sixteen instances exceeds the supported limit for data parallel training

No such limit. AWS benchmarks SMDDP to 128 nodes with near-linear scaling.

D Each instance trained on the full dataset, so the model saw the data sixteen times

A real misconfiguration when input is FullyReplicated rather than sharded. But that would slow each epoch, not cause non-convergence.

THE SIGNAL

The speed-up worked, so distribution is functioning. Only the optimisation changed, and it changed because the batch size did.

What to take from this module

- **Data parallel for speed. Model parallel for memory.** They are not interchangeable.
- **Distributing scales your batch size**, so the learning rate must be retuned.
- **Spot needs checkpoints, and the script must resume from them.** Both halves.
- **Warm pools: 60 minutes maximum, matching specs, billable while idle.**
- **Compare total job cost, not hourly rate.** Faster and dearer is often cheaper.

> ON THE EXAM
>
> Module 15 covers tuning what runs inside the job: hyperparameters, search strategies and regularisation.